

$n$ -element max-heap that runs in  $O(\lg n)$  time plus the overhead for mapping priority queue objects to array indices.

### 6.5-11

Give an  $O(n \lg k)$ -time algorithm to merge  $k$  sorted lists into one sorted list, where  $n$  is the total number of elements in all the input lists. (*Hint*: Use a min-heap for  $k$ -way merging.)

---

## Problems

### 6-1 *Building a heap using insertion*

One way to build a heap is by repeatedly calling MAX-HEAP-INSERT to insert the elements into the heap. Consider the procedure BUILD-MAX-HEAP' on the facing page. It assumes that the objects being inserted are just the heap elements.

```
BUILD-MAX-HEAP' ( $A, n$ )
1  $A.heap-size = 1$ 
2 for  $i = 2$  to  $n$ 
3   MAX-HEAP-INSERT( $A, A[i], n$ )
```

- a. Do the procedures BUILD-MAX-HEAP and BUILD-MAX-HEAP' always create the same heap when run on the same input array? Prove that they do, or provide a counterexample.
- b. Show that in the worst case, BUILD-MAX-HEAP' requires  $\Theta(n \lg n)$  time to build an  $n$ -element heap.

### 6-2 *Analysis of $d$ -ary heaps*

A  **$d$ -ary heap** is like a binary heap, but (with one possible exception) nonleaf nodes have  $d$  children instead of two children. In all parts of this problem, assume that the time to maintain the mapping between objects and heap elements is  $O(1)$  per operation.

- a. Describe how to represent a  $d$ -ary heap in an array.

- b.** Using  $\Theta$ -notation, express the height of a  $d$ -ary heap of  $n$  elements in terms of  $n$  and  $d$ .
- c.** Give an efficient implementation of EXTRACT-MAX in a  $d$ -ary max-heap. Analyze its running time in terms of  $d$  and  $n$ .
- d.** Give an efficient implementation of INCREASE-KEY in a  $d$ -ary max-heap. Analyze its running time in terms of  $d$  and  $n$ .
- e.** Give an efficient implementation of INSERT in a  $d$ -ary max-heap. Analyze its running time in terms of  $d$  and  $n$ .

### 6-3 Young tableaux

An  $m \times n$  **Young tableau** is an  $m \times n$  matrix such that the entries of each row are in sorted order from left to right and the entries of each column are in sorted order from top to bottom. Some of the entries of a Young tableau may be  $\infty$ , which we treat as nonexistent elements. Thus, a Young tableau can be used to hold  $r \leq mn$  finite numbers.

- a.** Draw a  $4 \times 4$  Young tableau containing the elements  $\{9, 16, 3, 2, 4, 8, 5, 14, 12\}$ .
- b.** Argue that an  $m \times n$  Young tableau  $Y$  is empty if  $Y[1, 1] = \infty$ . Argue that  $Y$  is full (contains  $mn$  elements) if  $Y[m, n] < \infty$ .
- c.** Give an algorithm to implement EXTRACT-MIN on a nonempty  $m \times n$  Young tableau that runs in  $O(m + n)$  time. Your algorithm should use a recursive subroutine that solves an  $m \times n$  problem by recursively solving either an  $(m - 1) \times n$  or an  $m \times (n - 1)$  subproblem. (*Hint:* Think about MAX-HEAPIFY.) Explain why your implementation of EXTRACT-MIN runs in  $O(m + n)$  time.
- d.** Show how to insert a new element into a nonfull  $m \times n$  Young tableau in  $O(m + n)$  time.
- e.** Using no other sorting method as a subroutine, show how to use an  $n \times n$  Young tableau to sort  $n^2$  numbers in  $O(n^3)$  time.

*f.* Give an  $O(m + n)$ -time algorithm to determine whether a given number is stored in a given  $m \times n$  Young tableau.

---

## Chapter notes

The heapsort algorithm was invented by Williams [456], who also described how to implement a priority queue with a heap. The BUILD-MAX-HEAP procedure was suggested by Floyd [145]. Schaffer and Sedgwick [395] showed that in the best case, the number of times elements move in the heap during heapsort is approximately  $(n/2) \lg n$  and that the average number of moves is approximately  $n \lg n$ .

We use min-heaps to implement min-priority queues in Chapters 15, 21, and 22. Other, more complicated, data structures give better time bounds for certain min-priority queue operations. Fredman and Tarjan [156] developed Fibonacci heaps, which support INSERT and DECREASE-KEY in  $O(1)$  amortized time (see Chapter 16). That is, the average worst-case running time for these operations is  $O(1)$ . Brodal, Lagogiannis, and Tarjan [73] subsequently devised strict Fibonacci heaps, which make these time bounds the actual running times. If the keys are unique and drawn from the set  $\{0, 1, \dots, n - 1\}$  of nonnegative integers, van Emde Boas trees [440, 441] support the operations INSERT, DELETE, SEARCH, MINIMUM, MAXIMUM, PREDECESSOR, and SUCCESSOR in  $O(\lg \lg n)$  time.

If the data are  $b$ -bit integers, and the computer memory consists of addressable  $b$ -bit words, Fredman and Willard [157] showed how to implement MINIMUM in  $O(1)$  time and INSERT and EXTRACT-MIN in  $O(\sqrt{\lg n})$  time. Thorup [436] has improved the  $O(\sqrt{\lg n})$  bound to  $O(\lg \lg n)$  time by using randomized hashing, requiring only linear space.

An important special case of priority queues occurs when the sequence of EXTRACT-MIN operations is *monotone*, that is, the values returned by successive EXTRACT-MIN operations are monotonically increasing over time. This case arises in several important applications, such as Dijkstra's single-source shortest-paths algorithm, which we discuss in Chapter 22, and in discrete-event simulation. For Dijkstra's

algorithm it is particularly important that the DECREASE-KEY operation be implemented efficiently. For the monotone case, if the data are integers in the range  $1, 2, \dots, C$ , Ahuja, Mehlhorn, Orlin, and Tarjan [8] describe how to implement EXTRACT-MIN and INSERT in  $O(\lg C)$  amortized time (Chapter 16 presents amortized analysis) and DECREASE-KEY in  $O(1)$  time, using a data structure called a radix heap. The  $O(\lg C)$  bound can be improved to  $O(\sqrt{\lg C})$  using Fibonacci heaps in conjunction with radix heaps. Cherkassky, Goldberg, and Silverstein [90] further improved the bound to  $O(\lg^{1/3+\epsilon} C)$  expected time by combining the multilevel bucketing structure of Denardo and Fox [112] with the heap of Thorup mentioned earlier. Raman [375] further improved these results to obtain a bound of  $O(\min \{\lg^{1/4+\epsilon} C, \lg^{1/3+\epsilon} n\})$ , for any fixed  $\epsilon > 0$ .

Many other variants of heaps have been proposed. Brodal [72] surveys some of these developments.

---

<sup>1</sup> In Python, dictionaries are implemented with hash tables.